

CUDA-METRO

GPU-Accelerated Metropolis Monte Carlo
for 2D Magnetic Materials

User Guide

A practical introduction for Python programmers

About this guide

This guide assumes you are comfortable writing Python scripts, using `numpy` arrays, and running programs from the terminal. It does *not* assume prior knowledge of GPU programming, CUDA, or condensed matter physics. Physics concepts are explained as needed.

Based on: Hait & Mahapatra (2025), *Journal of Open Source Software*, 10(109), 7589

CUDA-METRO — <https://github.com/arkavo/CUDA-METRO>

Contents

1	What Is CUDA-METRO?	3
1.1	High-level architecture	3
2	Physics Background (Short Version)	3
2.1	Spins on a lattice	3
2.2	The Hamiltonian (energy function)	3
2.3	Metropolis Monte Carlo	4
3	Installation and Dependencies	4
3.1	Requirements	4
3.2	Installing packages	5
3.3	Running scripts	5
4	Configuration File	5
4.1	The most important parameters	6
4.1.1	SIZE	6
4.1.2	Blocks	6
4.1.3	stability_runs and stability_wrap	6
4.1.4	FM_Flag	7
5	Material Parameters	7
5.1	CSV column order	7
5.2	Available materials	7
5.3	The neighbour structure descriptor (NBS)	7
6	Running a Simulation (Single Layer)	8
6.1	The minimal workflow	8
6.2	Step-by-step explanation	8
6.3	Choosing the right <code>run_mc_*</code> method	8
6.4	Understanding the output directory	9
7	The Spin Grid: Memory Layout	9
7.1	Single-layer layout	9
7.2	Multilayer layout	9
8	Multilayer Simulations	10
8.1	Physical picture	10
8.2	Running a multilayer simulation	10
8.3	Spin initialisation modes	11
8.4	Measuring magnetisation	11
9	Spatially Varying Magnetic Field Zones	11
9.1	How it works	11
9.2	API	11
9.3	Zone layout example	12
9.4	Number of zones is flexible	12
10	The Hysteresis Experiment	12

10.1	Field sweep protocol	13
10.2	Equilibration (EQ_WRAP)	13
10.3	Key parameters	13
10.4	Outputs	14
10.5	Magnetisation units	14
10.6	Loading saved data	14
11	Visualisation and Analysis	14
11.1	The Analyze class	14
11.1.1	Single-layer methods	15
11.1.2	Multilayer methods	15
11.2	Manual analysis example	15
12	Understanding CUDA Concepts (as a Python User)	15
12.1	GPU vs CPU memory	15
12.2	CUDA kernels	16
12.3	Pre-allocated random numbers	16
12.4	VRAM budget	16
13	Common Mistakes and How to Avoid Them	16
14	Performance Tuning	17
14.1	Parallelism ratio	17
14.2	Benchmark reference points	17
14.3	Tuning for a specific GPU	18
15	Quick-Reference Tables	18
15.1	MonteCarlo API	18
15.2	Config key summary	18
16	Worked Example: CrPS₄-ML Hysteresis	19
16.1	Setup	19
16.2	Field sweep construction	19
16.3	Temperature loop	19
17	Next Steps and Further Reading	20
17.1	Running your own material	20
17.2	Finding the Curie temperature	20
17.3	Cite the software	20
17.4	Reporting issues	20

1 What Is CUDA-METRO?

CUDA-METRO is a Python library for simulating the magnetic behaviour of thin 2D materials on an NVIDIA GPU. It answers questions like:

- At what temperature does a magnet lose its ordering (Curie temperature)?
- What spin texture forms when a material is cooled slowly?
- How does the magnetisation respond when you sweep an external field?

The simulation runs a **Metropolis Monte Carlo** algorithm: spins are proposed random moves and accepted or rejected based on energy. Running millions of such steps lets the system relax to a physically realistic state. Doing this on a GPU instead of a CPU makes it 100–1000× faster.

1.1 High-level architecture

File	Language	Role
<i>construct.py</i>	Python	User-facing API; orchestrates the simulation
<i>montecarlo.py</i>	Python + CUDA C	GPU kernels compiled at import time
<i>hysteresis.py</i>	Python	M_z – H hysteresis experiment
<i>multilayer.py</i>	Python	Multilayer equilibration with field zones
<i>visualize.py</i>	Python	Post-run spin visualisation
<i>inputs/TC-*.csv</i>	CSV	Material parameter database (>60 materials)

The two classes you will use are:

- **MonteCarlo** — creates and runs a simulation.
- **Analyze** — loads saved results and generates plots.

2 Physics Background (Short Version)

You do not need to be a physicist to use this software, but a little background helps you understand what the parameters mean.

2.1 Spins on a lattice

A magnetic material is modelled as a 2D grid of atoms. Each atom carries a **spin** — a tiny magnetic moment pointing in some direction in 3D space. We represent it as a 3-component vector: $\mathbf{S}_i = (S_x, S_y, S_z)$.

The magnitude $|\mathbf{S}_i|$ equals the **spin quantum number** S (e.g. $S = 3/2$ for Cr^{3+} ions in CrPS_4). At each MC step we propose a new random direction for one spin and decide whether to accept it.

2.2 The Hamiltonian (energy function)

The energy of spin i interacting with its neighbours is:

$$H_i = - \sum_j J_j \mathbf{S}_i \cdot \mathbf{S}_j - \sum_{j,\alpha} K_j^\alpha S_i^\alpha S_j^\alpha - A S_{iz}^2 - \sum_j \mathbf{D}_{ij} \cdot (\mathbf{S}_i \times \mathbf{S}_j) - \mu B \cdot S_{iz} \quad (1)$$

Term	Symbol	Physical meaning
Exchange	J_j	Tendency for neighbours to align ($J > 0$: ferromagnetic, all spins point the same way; $J < 0$: antiferromagnetic, alternating)
Anisotropic exchange	K_j^α	Direction-dependent version of exchange
Single-ion anisotropy	A	Preference for spins to point along a particular axis (easy-axis if $A < 0$, easy-plane if $A > 0$)
DMI	$\mathbf{D}_{ij}$	Dzyaloshinskii-Moriya interaction; causes spins to twist at an angle, stabilising skyrmions
External field	B	Applied magnetic field; pulls S_z toward alignment

2.3 Metropolis Monte Carlo

Monte Carlo simulation generates a sequence of spin configurations whose statistics match thermodynamic equilibrium at temperature T . The **Metropolis algorithm** achieves this with one simple rule per proposed move:

1. Pick a spin i at random.
2. Generate a trial spin direction $\mathbf{S}'_i$ uniformly on the unit sphere.
3. Compute the energy change $\Delta E = H(\text{trial}) - H(\text{current})$.
4. Accept the trial if $\Delta E < 0$ (lower energy is always better), *or* with probability $e^{-\Delta E/k_B T}$ even if $\Delta E > 0$ (thermal fluctuations can push the system uphill).

Repeating this millions of times lets the system find low-energy spin configurations. At high temperature, thermal energy is large so many uphill moves are accepted and the spins are disordered. At low temperature, almost only downhill moves are accepted and order builds up.

Why GPU? — Each proposed move only touches a small neighbourhood of spins, so many proposals can be evaluated *simultaneously* on different parts of the lattice. An NVIDIA GPU can run thousands of such threads in parallel, making the simulation orders of magnitude faster than CPU code.

3 Installation and Dependencies

3.1 Requirements

- NVIDIA GPU with CUDA support (any compute capability ≥ 5.0).
- CUDA Toolkit installed (version 11 or newer recommended).
- Python 3.9+.

- The following Python packages:

Package	Purpose
pycuda	Python bindings for CUDA; compiles and runs GPU kernels
numpy	Array operations on the CPU side
matplotlib	Plotting
seaborn	Heatmap visualisation
tqdm	Progress bars

3.2 Installing packages

```
pip install pycuda numpy matplotlib seaborn tqdm
```

pyCUDA requires a working CUDA installation. If `pip install pycuda` fails, check that `nvcc --version` works in your terminal. You may need to set `CUDA_ROOT` or install the CUDA toolkit from NVIDIA's website first.

3.3 Running scripts

All scripts are run from inside the `src/cudametro/` directory:

```
cd src/cudametro
python sample.py
```

4 Configuration File

Every simulation is controlled by a **JSON configuration file**. Here is a fully annotated example:

Listing 1: Annotated configuration file

```
{
  "Single_Mat_Flag" : 1,          // 1 = use a single material (
    default)
  "Animation_Flags" : 0,          // 0 = do not save animation
    frames
  "DMI_Flag"        : 1,          // 1 = include Dzyaloshinskii-
    Moriya term
  "TC_Flag"         : 0,          // 1 = sweep temperature to
    find Tc
  "Static_T_Flag"   : 1,          // 1 = fixed temperature; 0 =
    temperature sweep
  "FM_Flag"         : 0,          // 1 = start all spins pointing
    up (+z)
                                // 0 = start with random
                                spin directions
  "Input_flag"      : 0,          // 1 = load a saved spin state
    as starting point
  "Input_File"      : "Starting_States/state.npy",
```

```

"Temps"           : ["2.0"],      // temperature(s) in
    Kelvin
"Material"        : "CrPS4",      // material name (must
    match inputs/TC_*.csv)

"SIZE"           : 200,           // lattice side length; grid is
    SIZE x SIZE
"Box"            : 16,           // internal tiling parameter
"Blocks"         : 2048,         // parallel MC proposals per
    inner loop step
"Threads"        : 2,           // threads per block (always 2;
    do not change)

"B"              : "0.0",        // initial external magnetic
    field (normalised)

"stability_runs"  : 300000,      // inner loop length: sweeps
    per wrap pass
"calculation_runs" : 50,         // sweeps used for
    measurement averaging
"stability_wrap"  : 30,         // outer loop: how many wrap
    passes to run
"calculation_wrap" : 35,

"Cmpl_Flag"      : 1,           // 1 = complex (full
    anisotropic) Hamiltonian
"Prefix"         : "CrPS4"      // prefix for the output
    directory name
}

```

4.1 The most important parameters

4.1.1 SIZE

The lattice is a $\text{SIZE} \times \text{SIZE}$ grid. A 200×200 grid has 40 000 sites. Larger lattices capture long-range structures (e.g. large skyrmions) but use more GPU memory and take longer.

4.1.2 Blocks

Blocks is the number of independent spin proposals evaluated *simultaneously* on the GPU per inner-loop iteration. Larger values speed up each iteration but consume more GPU memory (the pre-allocated random number arrays scale as $\text{Blocks} \times \text{stability_runs}$).

A good rule of thumb is $\text{Blocks} \approx 0.1 \times \text{SIZE}^2$ (10% of all sites).

4.1.3 stability_runs and stability_wrap

The outer loop runs **stability_wrap** times. Each outer iteration runs **stability_runs** inner Metropolis sweeps and saves one grid snapshot. Total sweeps = $\text{stability_wrap} \times \text{stability_runs}$.

4.1.4 FM_Flag

- **FM_Flag = 1:** All spins start pointing along $+z$. Good for studying ferromagnets at low field.
- **FM_Flag = 0:** Spins start in random directions. Required when applying field zones (§9) to avoid the system being so stable that no moves are accepted.

5 Material Parameters

Material properties are stored in CSV files in the *inputs/* folder. The filename convention is *TC_MaterialName.csv*. The config field "Material": "CrPS4" instructs the code to load *inputs/TC_CrPS4.csv*.

5.1 CSV column order

Array index	Symbol	Description	Unit
0	S	Spin quantum number	—
1	J_1	1st-neighbour exchange	meV
2	J_2	2nd-neighbour exchange	meV
3	J_3	3rd-neighbour exchange	meV
4	J_4	4th-neighbour exchange	meV
5–7	$K_{1x,y,z}$	Anisotropic exchange, shell 1	meV
8–10	$K_{2x,y,z}$	Anisotropic exchange, shell 2	meV
11–13	$K_{3x,y,z}$	Anisotropic exchange, shell 3	meV
14–16	$K_{4x,y,z}$	Anisotropic exchange, shell 4	meV
17–19	$A_{x,y,z}$	Single-ion anisotropy	meV
20–23	NBS	Neighbour-shell descriptor	int
24	T_c	Reference Curie temperature	K
25	D	DMI magnitude	meV

5.2 Available materials

The *inputs/* directory includes over 60 parameterised materials, including CrI₃, CrBr₃, CrCl₃, CrPS₄, CrPS₄-ML, FePS₃, MnSe₂, VSe₂, and many more. Use "Material": "CrI3" in your config to load *inputs/TC_CrI3.csv*.

5.3 The neighbour structure descriptor (NBS)

The NBS field (indices 20–23) encodes how many neighbours exist at each shell distance. The code uses this to select the correct CUDA kernel:

NBS	Crystal	Typical materials
3-6-3-6	Honeycomb	CrI ₃ , CrPS ₄ , CrBr ₃
6-6-6-12	Triangular	CrTe ₂ , MnSe ₂
4-4-4-8	Square	CrO ₂ , MnO ₂
2-2-4-2	Rectangular	—
2-4-2-4	Rectangular variant	—

6 Running a Simulation (Single Layer)

6.1 The minimal workflow

```

1 import construct as cst
2 import numpy as np
3 from tqdm import tqdm
4
5 # 1. Create the simulation object from a config file
6 sim = cst.MonteCarlo(config="sample_config.json")
7
8 # 2. Initialise the spin grid
9 sim.mc_init()
10 sim.display_material() # print material parameters to
    terminal
11
12 # 3. Run the equilibration loop
13 for i in tqdm(range(sim.S_Wrap), desc="Stability Runs"):
14     sim.generate_random_numbers(sim.stability_runs)
15     grid = sim.run_mc_dmi_66612(sim.T[0]) # returns current
        grid
16     np.save(f"{sim.save_directory}/grid_{i:04d}", grid)
17
18 print("Done. Results saved in:", sim.save_directory)

```

Listing 2: Minimal single-layer simulation — sample.py

6.2 Step-by-step explanation

Step 1: `MonteCarlo(config=...)` Reads the JSON config, loads the material CSV, allocates all GPU memory, and creates a timestamped output directory (e.g. *Output_CrPS4_2025_01_15_14_3*). A *metadata.json* summary is written there automatically.

Step 2: `mc_init()` Allocates the spin grid on the CPU and GPU. If `FM_Flag=1`, all spins are initialised to $(S_x, S_y, S_z) = (0, 0, S)$. If `FM_Flag=0`, each spin is placed at a random point on the unit sphere and scaled by S .

Step 3: The outer loop Each iteration of the loop (controlled by `stability_wrap`) does:

1. `generate_random_numbers`: fill four GPU arrays with fresh uniform random floats (for site selection, trial spin direction, and Metropolis acceptance test).
2. `run_mc_*`: run `stability_runs` Metropolis sweeps on the GPU. Returns the updated spin grid as a flat `numpy` array.
3. `np.save`: write the current grid to disk for later visualisation.

6.3 Choosing the right `run_mc_*` method

Pick the method that matches your material's crystal structure:

Method	Crystal / Use
<code>run_mc_dmi_66612(T)</code>	Triangular lattice (6-6-6-12) with DMI
<code>run_mc_dmi_4448(T)</code>	Square lattice (4-4-4-8) with DMI
<code>run_mc_dmi_3636(T)</code>	Honeycomb lattice (3-6-3-6) with DMI
<code>run_mc_3636_multilayer(T, layers)</code>	Honeycomb multilayer

6.4 Understanding the output directory

After the run, the output folder contains:

- `metadata.json` — all simulation parameters.
- `grid_0000.npy`, `grid_0001.npy`, ... — one spin grid snapshot per `stability_wrap` iteration. Each is a flat `float32` array of length $\text{SIZE}^2 \times 3$.

7 The Spin Grid: Memory Layout

Understanding the grid layout is essential when you want to analyse or plot results yourself.

7.1 Single-layer layout

```

1 # grid is a 1D float32 array of length SIZE*SIZE*3
2 grid = sim.grid
3
4 # Index of spin component k at site (row, col):
5 # k=0 -> Sx, k=1 -> Sy, k=2 -> Sz
6 site = row * SIZE + col
7 value = grid[site * 3 + k]
8
9 # Or reshape to a 3D array for convenient access:
10 g = grid.reshape(SIZE, SIZE, 3)
11 Sz = g[:, :, 2] # 2D array of z-components

```

7.2 Multilayer layout

For L layers the grid is extended along a new outermost axis:

```

1 # grid is a 1D float32 array of length L*SIZE*SIZE*3
2 # Layer index is outermost:
3 pti = layer * SIZE*SIZE + row*SIZE + col
4 value = grid[pti * 3 + k]
5
6 # Or reshape to a 4D array:
7 g = grid.reshape(L, SIZE, SIZE, 3)
8 Sz_layer2 = g[2, :, :, 2] # z-component of layer 2

```

Flat memory is laid out as:

Layer 0 ($N^2 \times 3$ floats)	Layer 1	Layer 2	... Layer $L-1$
----------------------------------	---------	---------	-----------------

8 Multilayer Simulations

A multilayer system stacks L 2D honeycomb planes on top of each other. Spins within each layer interact via the usual in-plane exchange and DMI terms. Adjacent layers are coupled through an **interlayer exchange** J_c (stored in the CSV file).

8.1 Physical picture

... Layer $L - 1$ (top)
... Layer $L - 2$
$\vdots$
... Layer 1
... Layer 0 (bottom)

Each site in layer $\ell > 0$ feels an additional energy contribution from the spin directly below it (same row and column, layer $\ell - 1$) and above it (layer $\ell + 1$):

$$H_{\text{inter}} = J_c \mathbf{S}_{i,\ell} \cdot \mathbf{S}_{i,\ell-1} + J_c \mathbf{S}_{i,\ell} \cdot \mathbf{S}_{i,\ell+1}$$

For CrPS₄-ML, $J_c = -0.16$ meV (antiferromagnetic), meaning adjacent layers prefer to point in opposite directions. The competition between the ferromagnetic in-plane coupling and the antiferromagnetic interlayer coupling produces rich magnetic phase diagrams.

8.2 Running a multilayer simulation

```

1 import construct as cst
2 import numpy as np
3 from tqdm import tqdm
4
5 LAYERS = 8
6
7 sim = cst.MonteCarlo(config="multilayer_config.json")
8 sim.mc_init_multilayer(layers=LAYERS)      # note: different init
      call
9 sim.display_material()
10
11 # (Optional) apply a spatially varying field -- see Section 8
12 sim.set_field_zones(
13     boundaries=[10, 50, 250, 290],
14     fields=[0.0, 0.0, 0.6, 0.0, 0.0]
15 )
16
17 for i in tqdm(range(sim.S_Wrap), desc="Stability Runs"):
18     sim.generate_random_numbers_multilayer(None)      #
      multilayer version
19     grid = sim.run_mc_3636_multilayer(sim.T[0], layers=LAYERS)
20     np.save(f"{sim.save_directory}/grid_{i:04d}", grid)

```

Listing 3: multilayer.py workflow

The key differences from the single-layer workflow are:

1. Use `mc_init_multilayer(layers=L)` instead of `mc_init()`.
2. Use `generate_random_numbers_multilayer(None)` (the argument is unused but kept for API consistency).
3. Use `run_mc_3636_multilayer(T, layers=L)`.

8.3 Spin initialisation modes

FM_Flag	Starting state
1	Ferromagnetic: every spin in every layer is set to $(S_x, S_y, S_z) = (0, 0, S)$ — all pointing up. The system is strongly stable at low temperature; very few moves are accepted initially.
0	Random: each spin is placed at a uniformly random point on the unit sphere. The system starts disordered and must equilibrate. Required when applying field zones so that the spins have room to respond to the spatial field gradient.

8.4 Measuring magnetisation

```

1 # Returns (M_total, M_per_layer)
2 M_tot, M_layers = sim.get_magnetization(layers=LAYERS)
3
4 # M_tot: scalar, mean Sz / spin S, over all layers and sites
5 # M_layers: numpy array of shape (LAYERS,), per-layer average
6 print(f"Total Mz/Msat = {M_tot:.4f}")
7 for i, ml in enumerate(M_layers):
8     print(f"    Layer {i}: Mz/Msat = {ml:.4f}")

```

Only the z -component of the spin (S_z) is used, normalised by the spin magnitude S . This gives $M_z/M_{\text{sat}} \in [-1, +1]$.

9 Spatially Varying Magnetic Field Zones

By default, a single uniform external field is applied to the entire lattice. CUDA-METRO also supports **field zones**: different columns of the lattice can have different field values.

9.1 How it works

The GPU stores the field as an array of `SIZE` floats, one value per column. The Hamiltonian reads `b[col]` for each site, where `col = pti % SIZE`. This means field zones are strips running vertically (parallel to the row axis).

9.2 API

```

1 # Uniform field (backward-compatible):
2 sim.set_field(0.6)
3

```

```

4 # Spatially varying field:
5 # boundaries: list of column indices dividing the lattice into
   zones
6 # fields: one value per zone (must have len(boundaries) + 1
   entries)
7 sim.set_field_zones(
8     boundaries=[10, 50, 250, 290],    # 4 boundaries -> 5 zones
9     fields=[0.0, 0.0, 0.6, 0.0, 0.0]
10 )

```

9.3 Zone layout example

For a 300×300 lattice with boundaries=[10, 50, 250, 290]:

Zone 0	Zone 1	Zone 2	Zone 3	Zone 4
cols 0–9	cols 10–49	cols 50–249	cols 250–289	cols 290–299
$B = 0.0$	$B = 0.0$	$B = 0.6$	$B = 0.0$	$B = 0.0$
(buffer)	(buffer)	(active)	(buffer)	(buffer)

The outer buffer zones have zero field and act as a magnetically neutral boundary. The central zone carries the experiment field. This design prevents edge effects from corrupting the region of interest.

Always use FM_Flag = 0 (random start) with field zones. Starting from an all-up FM state with a positive z -field makes the initial configuration so energetically stable that the acceptance rate collapses to near zero and the simulation appears frozen.

9.4 Number of zones is flexible

The number of zones is not locked to 5. Provide any list of boundaries and a matching fields list:

```

1 # 3 zones (two buffers, one central)
2 sim.set_field_zones(
3     boundaries=[50, 150],
4     fields=[0.0, 1.0, 0.0]
5 )
6
7 # 7 zones
8 sim.set_field_zones(
9     boundaries=[20, 50, 100, 200, 250, 280],
10    fields=[0.0, 0.2, 0.4, 0.6, 0.4, 0.2, 0.0]
11 )

```

10 The Hysteresis Experiment

hysteresis.py measures the M_z – H hysteresis loop: how the magnetisation responds as the external field is swept up, then down, then back up again. This is the same

measurement made in a physical magnetometer.

10.1 Field sweep protocol

The field is swept in three branches:

$$0 \rightarrow +H_{\max} \rightarrow -H_{\max} \rightarrow +H_{\max}$$

```

1 h1 = np.linspace(0,          H_MAX,  N_STEPS,      endpoint=False)
  # ramp up
2 h2 = np.linspace(H_MAX,    -H_MAX,  2*N_STEPS,      endpoint=False)
  # down sweep
3 h3 = np.linspace(-H_MAX,   H_MAX,  2*N_STEPS,      endpoint=True)
  # up sweep
4 H_sweep = np.concatenate([h1, h2, h3])

```

At each field value:

1. `set_field(h_val)` updates the GPU field array.
2. `EQ_WRAP` passes of `generate + run_mc` equilibrate the spins at this field.
3. `get_magnetization()` records M_z/M_{sat} and M_z/M_{sat} per layer.

Crucially, the spin state is **not reset** between field steps. The configuration at field H_n seeds the simulation at H_{n+1} . This is what produces hysteresis: the system remembers which way it was magnetised.

10.2 Equilibration (EQ_WRAP)

`EQ_WRAP` is the number of `generate + run_mc` passes executed at each field point. More passes allow the spins more time to respond to a field change. The total number of Metropolis sweeps at each field point is:

$$\text{sweeps per field point} = \text{EQ_WRAP} \times \text{stability_runs}$$

At low temperatures (below ~ 5 K for CrPS₄-ML) thermal fluctuations are rare and the system needs more sweeps to overcome local energy barriers. Increasing `EQ_WRAP` does not use extra GPU memory; it only adds compute time.

10.3 Key parameters

Parameter	Default	Meaning
LAYERS	12	Number of magnetic layers
TEMPERATURES	[0.1, 0.5, 1, 2, 5, 10, 15, 20] K	Temperatures to simulate
H_MAX	1.5	Maximum normalised field
N_STEPS	4	Field points per branch
EQ_WRAP	10	Equilibration passes per field point

10.4 Outputs

All outputs are saved in a timestamped directory *Hysteresis_YYYY_MM_DD_HH_MM_SS/*:

File	Content
<i>hysteresis_data.npy</i>	Raw data dictionary; keys are temperatures; each entry contains arrays <i>H</i> , <i>M_total</i> , <i>M_layers</i> , and <i>snapshots</i>
<i>MH_all_temps.png</i>	M_z/M_{sat} and emu/g vs H for all temperatures; line style encodes branch
<i>MH_per_layer-T{X}K.png</i>	Per-layer M_z vs H at each temperature
<i>spinstate-T{X}K-H0.png</i>	Heatmaps of S_x , S_y , S_z components at $H \approx 0$ for each temperature

10.5 Magnetisation units

Magnetisation is reported in two units side by side:

- **Normalised:** $M_z/M_{\text{sat}} \in [-1, +1]$, where +1 means all spins fully aligned along + z .
- **emu/g:** the physical unit used in magnetometry. For CrPS₄:

$$M_{\text{sat}}^{\text{emu/g}} = \frac{g \cdot \mu_B \cdot S \cdot N_A}{M_{\text{molar}}} = \frac{2.0 \times 9.274 \times 10^{-21} \times 1.5 \times 6.022 \times 10^{23}}{211.21} \approx 79.3 \text{ emu/g}$$

10.6 Loading saved data

```

1 import numpy as np
2
3 data = np.load("Hysteresis_2025_01_15_14_30_00/hysteresis_data.
4               npy",
5               allow_pickle=True).item()
6
7 # data is a dict keyed by temperature (float)
8 T = 2.0
9 H = data[T]["H"]           # field sweep array
10 M = data[T]["M_total"]     # total Mz/Msat at each field point
11 Ml = data[T]["M_layers"]   # shape (N_H, LAYERS): per-layer Mz
12 snaps = data[T]["snapshots"] # {H_value: grid_array}

```

11 Visualisation and Analysis

11.1 The Analyze class

```

1 viewer = cst.Analyze(
2     directory="Output_CrPS4_2025_01_15_14_30_00",
3     reverse=False,      # False = plot frames in forward order
4 )

```

11.1.1 Single-layer methods

```

1 viewer.spin_view()          # heatmaps of  $S_x$ ,  $S_y$ ,  $S_z$  for each
    saved grid
2 viewer.quiver_view()        # in-plane ( $S_x$ ,  $S_y$ ) arrows coloured by
     $S_z$ 

```

11.1.2 Multilayer methods

```

1 viewer.spin_view_multilayer(layers=8)    # 3 x L heatmap grid
2 viewer.quiver_view_multilayer(layers=8)  # quiver per layer,
    shared  $S_z$  colour

```

11.2 Manual analysis example

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 SIZE = 300
5 LAYERS = 8
6
7 # Load the last saved grid
8 grid = np.load("Output_CrPS4/grid_0029.npy")
9 g = grid.reshape(LAYERS, SIZE, SIZE, 3)
10
11 fig, axes = plt.subplots(1, LAYERS, figsize=(4*LAYERS, 4))
12 for i, ax in enumerate(axes):
13     ax.imshow(g[i, :, :, 2], cmap="coolwarm", vmin=-1.5, vmax=
        =1.5)
14     ax.set_title(f"Layer {i}")
15     ax.axis("off")
16 plt.suptitle("Sz per layer")
17 plt.tight_layout()
18 plt.savefig("sz_layers.png")

```

12 Understanding CUDA Concepts (as a Python User)

You do not need to write any CUDA code yourself. But understanding what happens behind the scenes helps you tune performance and avoid common mistakes.

12.1 GPU vs CPU memory

A GPU has its own memory (VRAM), separate from the computer's regular RAM. Data must be explicitly copied between them. In pyCUDA:

```

1 # CPU -> GPU (host to device)
2 drv.memcpy_htod(GPU_array, cpu_numpy_array)
3
4 # GPU -> CPU (device to host)
5 drv.memcpy_dtoh(cpu_numpy_array, GPU_array)

```


CUDA-METRO handles all of this for you inside `MonteCarlo`. When you call `run_mc_*`, the function launches the GPU kernel, lets it run, then copies the result back to `sim.grid` on the CPU.

12.2 CUDA kernels

A CUDA kernel is a function that runs on the GPU, executed simultaneously by many threads. Each thread handles one spin proposal. The two threads in each block in CUDA-METRO work as a pair: one computes the energy of the current spin, the other computes the energy of the trial spin. They synchronise, compare, and write the result.

12.3 Pre-allocated random numbers

Generating random numbers on the GPU is expensive. CUDA-METRO pre-allocates eight large GPU arrays at `mc_init_multilayer` time:

$$\text{size of each array} = \text{Blocks} \times \text{stability_runs}$$

Each outer loop iteration calls `rg.fill_uniform()` to refill these in place (no new allocation). This is why changing `Blocks` or `stability_runs` after initialisation is not supported — the array sizes are fixed at init time.

12.4 VRAM budget

```
1 import pycuda.driver as drv
2 free, total = drv.mem_get_info()
3 used_mb = (total - free) / 1024**2
4 print(f"VRAM used: {used_mb:.1f} MB")
```

The dominant allocation is the random number arrays. For `Blocks = 4096`, `stability_runs = 60000`:

$$8 \text{ arrays} \times 4096 \times 60000 \times 4 \text{ bytes} \approx 7.5 \text{ GB}$$

On a GPU with 32 GB VRAM, you have room to push this further.

13 Common Mistakes and How to Avoid Them

1. Simulation appears frozen / no spin changes

Cause: `FM_Flag = 1` combined with a field in the $+z$ direction. The system is fully polarised and every trial spin (random direction) has higher energy — almost no moves are accepted.

Fix: Set `FM_Flag = 0` in the config to start from a random state, especially when using field zones or studying antiferromagnets.

2. Wrong number of layers initialised

If you call `mc_init_multilayer(layers=8)` but your config was designed for 4 layers, the spin grid will have 8 layers but the saved metadata will say 4. Always keep `LAYERS` consistent between the Python script and the config file.

3. VRAM out-of-memory error

Cause: `Blocks × stability_runs` is too large.

Fix: Reduce one of them. A useful diagnostic:

```
1 import pycuda.driver as drv
2 free, total = drv.mem_get_info()
3 print(f"Free VRAM: {free/1e9:.1f} GB")
```

4. Magnetisation does not saturate at low temperature

Cause: Too few equilibration passes. At very low T , the system needs more sweeps to tunnel out of local energy minima.

Fix: Increase `EQ_WRAP` (hysteresis) or `stability_wrap` (multilayer) at the cost of longer run time.

5. Wrong material loaded

The config field "Material" must exactly match the CSV filename stem (case-sensitive on Linux). For `inputs/TC_CrI3.csv`, use "Material": "CrI3".

6. ImportError when running outside `src/cudametro/`

`construct.py` uses relative imports. Always run scripts from `src/cudametro/` or add the directory to `sys.path`:

```
1 import sys
2 sys.path.insert(0, "/path/to/src/cudametro")
3 import construct as cst
```

14 Performance Tuning

14.1 Parallelism ratio

The fraction of sites updated per inner step is:

$$f = \frac{\text{Blocks}}{\text{SIZE}^2}$$

Higher f speeds up convergence per wall-clock second but reduces accuracy (the strict Metropolis detailed-balance condition is violated more). A ratio of $f \approx 0.1$ (10% of sites per step) is the recommended default, giving accuracy comparable to single-spin-update algorithms.

14.2 Benchmark reference points

Lattice	GPU	Time per full run
200×200	NVIDIA V100	~30 s
500×500	NVIDIA A100	~300 s
750×750	NVIDIA A100	~9 h

14.3 Tuning for a specific GPU

If you have a GPU with large VRAM (e.g. 32 GB), you can increase `stability_runs` and `Blocks` to run more sweeps per outer iteration without saving to disk as often. This reduces I/O overhead for very long runs.

15 Quick-Reference Tables

15.1 MonteCarlo API

Method	Description
<code>MonteCarlo(config)</code>	Constructor. Reads config, loads material, allocates GPU memory.
<code>mc_init()</code>	Initialise single-layer spin grid.
<code>mc_init_multilayer(layers=L)</code>	Initialise multilayer spin grid.
<code>display_material()</code>	Print material parameters to terminal.
<code>generate_random_numbers(n)</code>	Single-layer: generate RNG arrays.
<code>generate_random_numbers_multilayer(n)</code>	Multilayer: refill pre-allocated RNG arrays.
<code>run_mc_dmi_66612(T)</code>	Run MC for triangular (6-6-6-12) crystal.
<code>run_mc_dmi_4448(T)</code>	Run MC for square (4-4-4-8) crystal.
<code>run_mc_dmi_3636(T)</code>	Run MC for honeycomb (3-6-3-6) crystal.
<code>run_mc_3636_multilayer(T, layers)</code>	Run MC for multilayer honeycomb system.
<code>set_field(B)</code>	Broadcast a uniform field to all columns.
<code>set_field_zones(boundaries, fields)</code>	Set per-column spatially varying field.
<code>get_magnetization(layers)</code>	Return (M_z/M_{sat} , M_z per layer).
<code>grid_reset()</code>	Re-initialise the spin grid from scratch.
<code>cleanup()</code>	Free GPU buffers.

15.2 Config key summary

Key	Type	Effect
<code>SIZE</code>	int	Lattice side length ($N \times N$)
<code>Blocks</code>	int	Parallel MC proposals per step
<code>stability_runs</code>	int	Sweeps per outer loop iteration
<code>stability_wrap</code>	int	Number of outer loop iterations
<code>FM_Flag</code>	0/1	0=random start; 1=all-up FM start
<code>DMI_Flag</code>	0/1	Include DMI term in Hamiltonian
<code>TC_Flag</code>	0/1	1=temperature sweep (T_c mode)
<code>Static_T_Flag</code>	0/1	1=fixed T; 0=linspace to $2T_c$
<code>Temps</code>	list	List of temperatures as strings
<code>Material</code>	string	Stem of CSV file in inputs/
<code>B</code>	string	Initial uniform field
<code>Prefix</code>	string	Output directory prefix

16 Worked Example: CrPS₄-ML Hysteresis

This example walks through *hysteresis.py* line by line.

16.1 Setup

```

1 # Physical constants for emu/g conversion
2 MU_B_CGS = 9.2741e-21 # erg/G (CGS Bohr magneton)
3 N_A      = 6.02214e23
4 G_LANDE  = 2.0
5 SPIN     = 1.5 # Cr3+ has S = 3/2
6 M_MOLAR  = 51.996 + 30.974 + 4*32.06 # CrPS4 = 211.21 g/mol
7 EMU_G_SAT = G_LANDE * MU_B_CGS * SPIN * N_A / M_MOLAR # ~79.3
              emu/g

```

16.2 Field sweep construction

```

1 H_MAX    = 1.5 # maximum field (in units of spin magnitude)
2 N_STEPS  = 4   # field points per branch
3
4 h1 = np.linspace(0, H_MAX, N_STEPS, endpoint=False)
5 h2 = np.linspace(H_MAX, -H_MAX, 2*N_STEPS, endpoint=False)
6 h3 = np.linspace(-H_MAX, H_MAX, 2*N_STEPS, endpoint=True)
7 H_sweep = np.concatenate([h1, h2, h3])
8 # Total: N_STEPS + 2*N_STEPS + 2*N_STEPS = 5*N_STEPS = 20 field
      points

```

16.3 Temperature loop

```

1 TEMPERATURES = [0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 15.0, 20.0] # K
2 EQ_WRAP      = 10 # equilibration passes per field point
3
4 for T in TEMPERATURES:
5     sim = cst.MonteCarlo(config="hysteresis_config.json")
6     sim.mc_init_multilayer(layers=LAYERS) # fresh random
        state each T
7
8     M_total_list, M_layers_list = [], []
9     snapshots = {}
10
11     for idx, h_val in enumerate(H_sweep):
12         sim.set_field(h_val)
13
14         # Equilibrate at this field
15         for _ in range(EQ_WRAP):
16             sim.generate_random_numbers_multilayer(None)
17             sim.run_mc_3636_multilayer(T, layers=LAYERS)
18

```

```

19         # Measure
20         M_tot, M_lay = sim.get_magnetization(LAYERS)
21         M_total_list.append(M_tot)
22         M_layers_list.append(M_lay.copy())
23
24         # Save a snapshot at  $H \sim 0$ 
25         if idx in SNAP_INDICES:
26             snapshots[h_val] = sim.grid.copy()

```

Why a fresh instance per temperature? Each temperature sweep creates a new MonteCarlo object. This resets the spin state to a fresh random configuration so that different temperatures are independent experiments. The GPU memory from the previous instance will be freed when the Python object goes out of scope.

17 Next Steps and Further Reading

17.1 Running your own material

1. Find or compute the exchange parameters $J_1, J_2, \dots$, DMI D , and anisotropy A for your material (typically from DFT calculations or literature).
2. Create a CSV file in *inputs/* following the format in Section 4.
3. Set "Material": "YourMaterial" in the config.
4. Adjust "SIZE" and "Blocks" for your GPU.
5. Run *sample.py* or write your own driver script.

17.2 Finding the Curie temperature

Set "TC_Flag": 1 and "Static_T_Flag": 0 in the config. The code will sweep temperature from near 0 to $2T_c$ and return $\langle M \rangle$ and the susceptibility χ at each temperature. The Curie temperature is identified as the peak of χ .

17.3 Cite the software

If you use CUDA-METRO in a publication, please cite:

Hait & Mahapatra (2025). *CUDA-METRO: A GPU-Accelerated Metropolis Monte Carlo Simulator for 2D Magnetic Materials*. Journal of Open Source Software, 10(109), 7589.

17.4 Reporting issues

Bugs and feature requests can be filed at the project's GitHub repository.